

- [实验目的](#)
- [预习内容](#)
- [实验内容](#)
 - [申请云服务器帐号](#)
 - [搭建VPN网络](#)
 - [选择Server节点](#)
 - [Server/Client的Interface配置](#)
 - [检查内核版本](#)
 - [安装相应的库](#)
 - [生成通讯需要的公私钥](#)
 - [配置Interface](#)
 - [Server节点配置防火墙协议](#)
 - [Server/Client的Peer配置](#)
 - [配置P2P通讯](#)
 - [Server P2P 通讯配置](#)
 - [配置Server负责转发Client间的通讯](#)
 - [Client P2P 通讯配置](#)
 - [测试](#)
 - [预期结果](#)
 - [基于QUIC+MQTT协议的打地鼠程序](#)
 - [架构简介](#)
 - [Docker简介](#)
 - [EMQX Broker的角色](#)
 - [NanoMQ客户端的作用](#)
 - [配置QUIC+MQTT所需环境](#)
 - [基于VPN和QUIC+MQTT的打地鼠程序](#)
 - [TCP和QUIC协议的对比](#)
 - [验收要求](#)
 - [参考](#)

实验目的

1. 熟悉云服务器的使用方法。
2. 熟悉RDK与云服务器之间基于 VPN 的安全通信方式。
3. 掌握RDK和云服务器之间基于 QUIC 协议通信的编程方法。

预习内容

1. 结合[参考代码](#)，熟悉Wireguard和NanoSDK的使用。
2. 准备好实验内容中所需的代码。

实验内容

申请云服务器帐号

本实验及实验五均需要一台具有公网IP的主机，这里主要展示阿里云服务器的申请和使用方法。

登陆阿里云官网的[免费试用界面](#)，选择云服务器ECS的免费试用，点击立即试用：

免费试用

立即领用云产品，开启云上实践之旅 [试用规则](#)

ECS热门应用 | 解决 Guestosssh异常

领取ECS免费资源，快速解决操作系统异...

"百搭ECS"系列，开发者实战指南

领取免费ECS试用资源，玩转各类热门场...

类目筛选

清除筛选

可试用人群

☐ 企业认证

☐ 个人认证

产品类别

计算

容器

存储

网络与CDN

安全

中间件

开发工具

迁移与运维管理

数据库

大数据计算

人工智能和机器学习

搜索试用产品

为您展示 158 款试用产品

云服务器 个人认证 额度3个月内有效

云服务器 ECS

专为中小应用打造，可轻松满足开发测试、网站测试和小型应用构建等场景需求；提供宝塔/LA...

每月免费额度2...

规格信息：①e系列 2核 2GB或者2核 4GB（每月200元免费额度）；②公网流量每月80元免费额度（可用于抵扣100GB国内地域流量）

试用教程 立即试用

Serverless 额度3个月内有效

函数计算 FC

事件驱动的全托管 Serverless 计算服务，无需管理服务器等基础设施，即可以弹性、可靠的方式运...

180元额度 3个月

规格信息：函数计算180元试用额度内可用，超出按量计费。试用额度可抵扣确定数额的活跃 vCPU 使用量，内存使用量，GPU 使用量以及函数调用次数。此外，

试用教程 立即试用

无影 个人认证 额度3个月内有效

无影云电脑（专业版）

云上超级电脑：安全便捷，适合办公、研发、教育等多场景使用

4核8GB 800...

规格信息：含800小时开机时长，100Gib 磁盘及网络。开通后默认开机，时长用尽/时长包到期未加购，资源将释放；商品提到的“核”指 vCPU

试用教程 立即试用

需要注册或登陆阿里云帐号，每一个帐号有3个月的试用期。之后可以试用服务器并选择相应的配置。一个参考配置如下：

云服务器实例、系统盘、系统及镜像

产品配置

2核(vCPU) 2 GiB

规格族

经济型e系列

系统盘

40 GiB ESSD Entry 云盘

2核(vCPU) 4 GiB

规格族

经济型e系列

系统盘

40 GiB ESSD Entry 云盘

可试用台数

1台

2台

试用额度

试用有效期	免费范围	超额情况
3个月	每月可免费使用约786小时 最高免费额度：¥200/月 且 ¥1/小时 北京、杭州、广州、成都、乌兰察布	超出免费额度部分需自付 按量收费>

基于节省计划的免费试用版本支持，灵活创建和弹性调整试用配置

操作系统

Alibaba Cloud Linux

3.2104 LTS 64位

Ubuntu

22.04 64位 UEFI版

Windows Server

2022 数据中心版 64位中文版

CentOS

7.9 64位 SCC版

预装应用

宝塔Linux面板

Linux版本，可一键部署LAMP/LNM
P/Tomcat/Node.js等应用

WordPress

包含Alibaba Cloud Linux3+Nginx
+MySQL5.7+PHP7.4+Wordpress
架构

LAMP

包含CentOS7.9+Apache+PHP7.4
+MySQL5.7架构，支持多版本切换

产品所在地域

华北 2（北京）

试用产品均为随机分配，如有需求，您可自定义选择地域

到期释放设置

现在设置。试用3个月到期前1小时，自动释放实例，ECS实例释放后数据不保留

如您试用过程中想取消释放，可登录控制台"释放设置-取消释放"。

暂不设置。试用到期前再设置释放，请注意试用到期未释放可能产生欠费

系统将在您的试用到期前，通过短信、邮件、控制台提醒您操作释放。

公网流量

流量试用额度

试用有效期	免费额度	超额情况
3个月	80元/月 每月可抵扣100GB国内地域流量	超出免费额度部分需自付 CDT产品计费>

基于云数据传输CDT支持免费额度，适用于ECS、传统型负载均衡CLB等多个产品，[CDT支持产品>](#)

注意事项

1.如果免费额度当月未用完，额度会在月末清零，不能延期到下个月。首月开通试用时长不满1个月的，额度仍可继续使用，[图文说明>](#)

2.当前试用产品不满足国内ICP备案要求，如需备案请前往购买页选购包年包月实例，[立即前往>](#)

协议

我同意，开通免费试用，即3个月免费节省计划，用于抵扣页面指定配置的按量云服务器ECS及ECS访问公网产生的流量

我同意，《云服务器试用须知》、《云服务器 ECS 服务条款》、《云数据传输CDT服务协议》



第一次使用服务器时，需要配置服务器的连接方式和密码。可以通过推荐的Workbench远程连接配置服务器的连接情况：





在这里记录服务器的公网ip并 **点击“重置密码”** 设置服务器中root用户名的密码。后续可以通过root用户名ssh远程登陆服务器，不需要再登陆网页访问服务器。

注：阿里云服务器默认禁止root用户密码登录；可参照[通过阿里云Workbench添加公钥](#)，在第一次通过 Workbench 连接服务器时添加公钥，之后可以通过公钥登录服务器。

可以选择在云服务器上建立自己的用户，防止配置错误导致环境混乱：

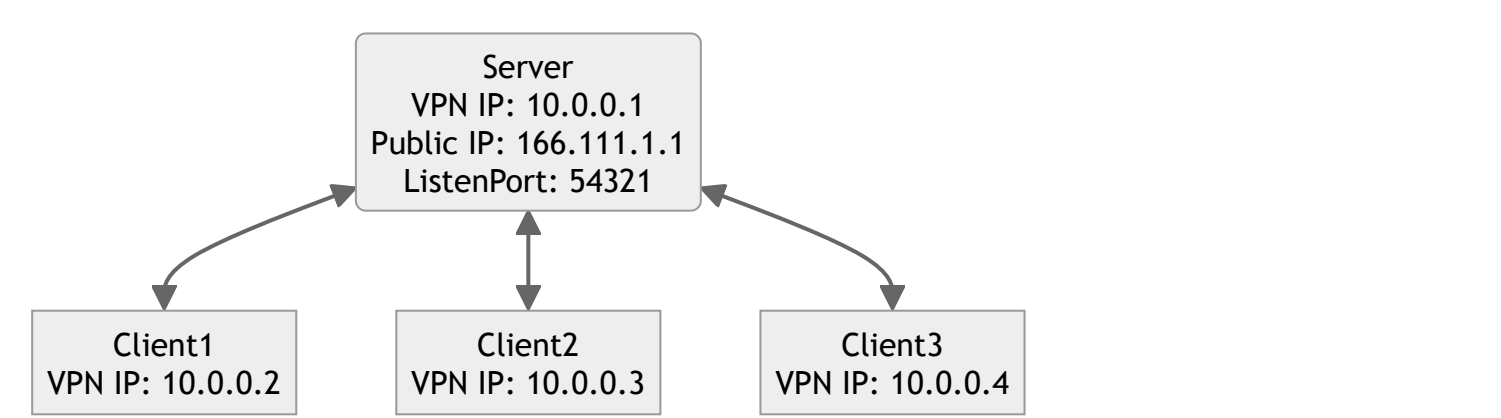
```
sudo useradd -m <your_username>
sudo usermod -aG sudo <your_username> # 添加sudo权限
```

搭建VPN网络

VPN主要用于解决多个节点不在同一个子网内部时，希望搭建一个虚拟的子网，将各个节点连接在一起，实现节点和节点之间直接互相访问和通信。

Wireguard是Linux内核自带的一个包，经过简单配置后，可以搭建一个VPN网络。在该VPN网络中，所有节点在拥有原本IP地址的基础上，将额外拥有一个属于本VPN的虚拟IP地址。本教程中，该虚拟IP地址为10.0.0.x。搭建VPN后，VPN内的所有节点（包括Server和Client）将处在同一个虚拟子网中，可以将前面几个实验中子网内部的操作迁移至该虚拟子网中。

本教程讲解如何利用Wireguard组建如下的VPN网络：



其中，VPN通常由若干Server节点和若干个Client节点组成，这里我们考虑一个简化版本，即一个Server节点和若干个Client节点。Server节点需要拥有公网IP，方便被其余的Client节点访问。Client节点之间的通信实际由Server节点负责转发。本实验中的Client为所有的RDK节点。下面将详细介绍各个节点本身的配置，及节点和节点之间连接的配置。

选择Server节点

由于Server需要能够被Client访问到，所以需要有公网IP。Server可以是RDK可以是任意安装了Linux且具有公网IP的计算机。本教程基于上文搭建的阿里云服务器进行介绍。

Server/Client的Interface配置

下面是使用wireguard时，Server和Client各自需要的基础配置，需要在Server和每个Client上都完成一遍。

检查内核版本

```
uname -a
```

确保Linux内核版本 >= 5.6

安装相应的库

```
sudo apt install wireguard-tools
```

生成通讯需要的公私钥

```
wg genkey | sudo tee /etc/wireguard/private.key # 生成私钥，保存在/etc/wireguard/private.key
sudo cat /etc/wireguard/private.key | wg pubkey | sudo tee /etc/wireguard/public.key #
```

上述生成的节点私钥存在/etc/wireguard/private.key, 公钥存在/etc/wireguard/public.key，两者均可通过 `cat` 指令查看（需要 [root 权限](#)）。

配置Interface

编辑/etc/wireguard/wg0.conf文件（参见[Vim教程](#)），填入以下内容：

```
[Interface]
Address = #自身节点在vpn中的ipv4地址
ListenPort = #自身节点监听端口
PrivateKey = #自身节点私钥
```

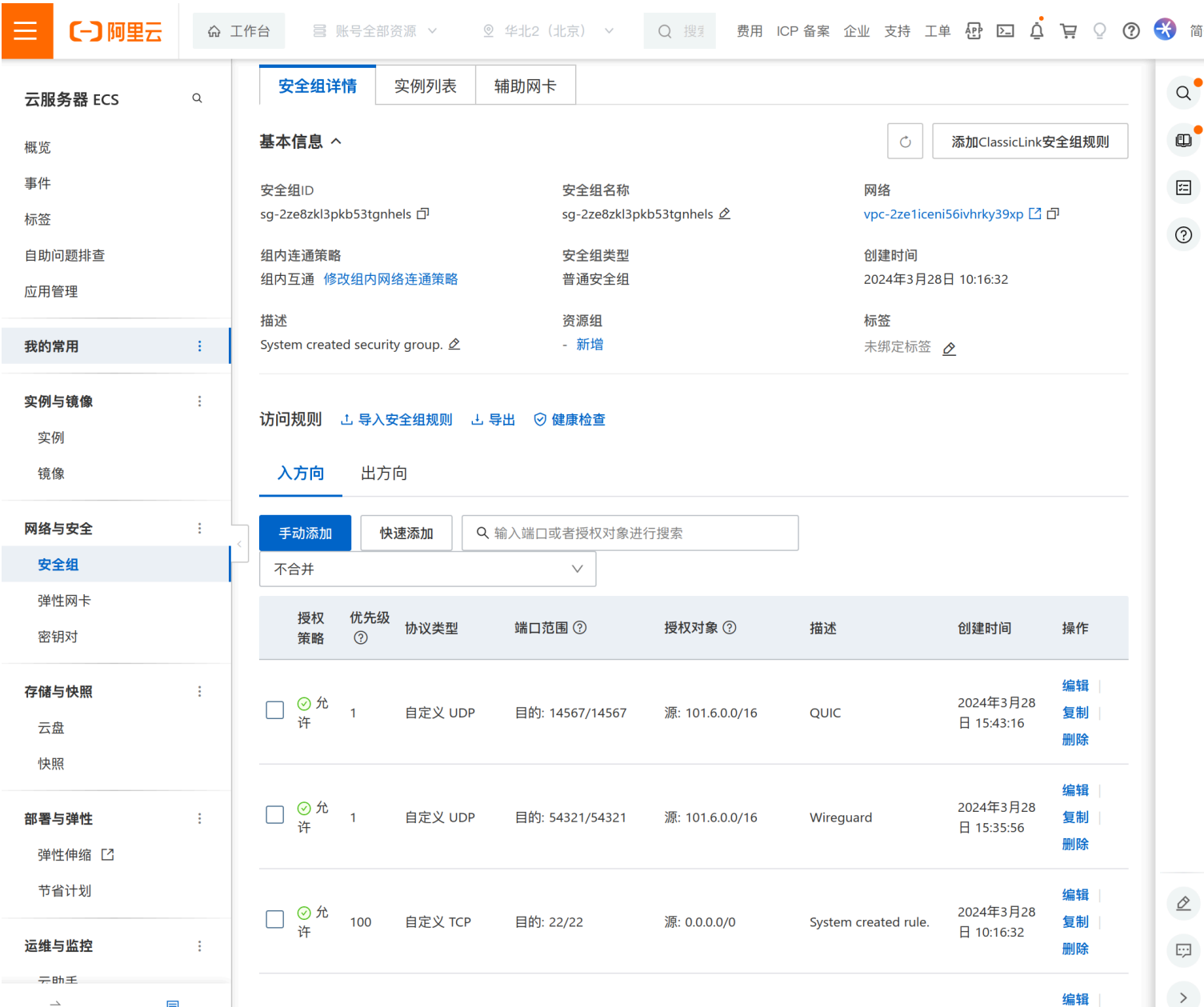
以前面的拓扑图为例

1. 每个节点的Address为10.0.0.x，方便起见，下文均假设Server的地址通常为10.0.0.1，Client的地址通常为10.0.0.x。
2. Server监听端口(ListenPort)为54321，其他client端也需要设置ListenPort
3. PrivateKey中填入上一步生成的私钥，可以用 `cat` 指令输出私钥文件中的内容。安全起见，私钥的内容注意保密。

Server节点配置防火墙协议

被连接的Server需要保证对应的监听端口的畅通。例如如果以阿里云作为服务器，则需要保证阿里云防火墙开放对应端口的UDP通讯。阿里云服务器开放监听端口需要通过阿里云网页配置。

在阿里云的网页中找到云服务器ECS的管理页面，在左侧边栏中找到网络与安全-安全组，手动添加规则，让Wireguard的端口（本样例中为54321端口）的入方向允许通过：



注：

- 安全组可能有多个，须选择当前服务器实例在用的安全组。例如，在下图中，安全组的 IP 占用数为 1，表明它正在被我们刚刚创建的服务器实例所使用。点击此条目右侧的“管理规则”即可。



- Wireguard 使用UDP协议，因此添加时需要选择UDP端口。
- 设置授权对象时，需选择允许访问云服务器54321端口的网段。例如，实验室有线网的网段为 166.111.0.0/16 。若需要从其他IP访问，可以按需添加。

Server/Client的Peer配置

单个节点配置好后，需要配置节点之间的连接关系。请将所有节点配置完上一部分后再开始这一部分的配置。

配置P2P通讯

该步用于建立Server和Client之间的连接关系，需要在Server的配置文件中加入Client的信息，在Client的配置文件中加入Server的信息。

Server P2P 通讯配置

编辑Server的/etc/wireguard/wg0.conf文件，在[Interface]后面加入各个Client的信息。

```
[Peer] #第一个Client 节点
PublicKey = #Client 节点的公钥
AllowedIPs = #Client 节点vpn内ip

[Peer] #第二个Client 节点
PublicKey =
AllowedIPs =

...
```

注意：

1. Client节点的公钥仍然是**生成通讯需要的公私钥**步骤中生成的Client节点的公钥，可以利用 `cat` 指令读取公钥文件中的内容，注意不要放成私钥了。
2. AllowedIPs可以被认为是本地的路由表。如果目标ip的范围落在这个里面，则包会发向这个对方节点。这里填写相应的Client的地址，如10.0.0.x。

配置Server负责转发Client间的通讯

Server需要打开内核的ipforward

```
sysctl -w net.ipv4.ip_forward=1
```

Client P2P 通讯配置

下面配置每个Client和Server的连接关系，下面的操作需要对每个Client都进行一遍。编辑Client的/etc/wireguard/wg0.conf文件，在[Interface]后面加入各个Server的信息。

```
[Peer]
PublicKey = #Server 节点的公钥
AllowedIPs = #需要通过Server 节点访问的ip 地址范围
Endpoint = #Server 节点的公网ip地址: 监听端口号
PersistentKeepalive =
```

注意：

1. Server节点的公钥仍然是**生成通讯需要的公私钥**步骤中生成的公钥，可以利用 `cat` 指令读取公钥文件中的内容，注意不要放成私钥了。
2. AllowedIPs可以被认为是本地的路由表。如果目标ip的范围落在这个里面，则包会发向这个对方节点。这个地方填写需要通过Server转发的IP地址网段；在我们的例子中即整个 VPN 网段，亦即 10.0.0.0/24。
3. PersistentKeepalive = x，则每x秒发送一个包给对方节点，用于保持链接。建议没有公网ip的Client设置Server的 PersistentKeepalive = 25，来确保Server和Client的链接不被掐断。

测试

1. 通过如下命令开启wireguard(假设之前配置文件为wg0.conf)

```
sudo wg-quick up wg0
```

2. 通过如下命令查看链接状态

```
wg
```

3. 如果已经连接，可以尝试ping vpn地址，并通过 `mtr` 或 `traceroute` 检查包通过的路径。

注：如果发现配置有误，更改配置后可通过如下命令重启wireguard

```
sudo wg-quick down wg0 && sudo wg-quick up wg0
```

预期结果

在开头图中的拓扑下，可以通过任意一个client ping到server 10.0.0.1。任意两个client之间也可以互相ping通。

两个client之间通信应该只经过两跳，比如下图所示：

```
[➔ ~ traceroute 10.0.0.3
traceroute to 10.0.0.3 (10.0.0.3), 64 hops max, 52 byte packets
 1  10.0.0.1 (10.0.0.1)  355.753 ms  268.874 ms  111.655 ms
 2  10.0.0.3 (10.0.0.3)  78.966 ms  76.788 ms  208.838 ms
```

基于QUIC+MQTT协议的打地鼠程序

下面将基于上面构建的VPN组网，通过Quic协议通信实现打地鼠程序。

架构简介

本实验通过基于QUIC协议的MQTT通信实现一个分布式打地鼠游戏。主要技术组件包括在Docker容器中运行的EMQX Broker以及部署在RDK上的NanoMQ客户端。

Docker简介

Docker是一个帮助软件运行在不同电脑上的工具。它可以将软件和它的依赖打包起来，确保这个软件无论在哪里运行都能表现一致。这样，我们可以轻松地多种电脑上运行相同的软件，简化安装和设置流程。

EMQX Broker的角色

EMQX Broker是一个MQTT消息代理服务器，部署在云服务器上，负责管理和转发来自各个客户端的消息。使用QUIC协议可以提高数据传输速度和降低延迟。

NanoMQ客户端的作用

NanoMQ是一个轻量级MQTT客户端，适用于资源受限的设备，如RDK。在本实验中，它负责发布和订阅消息，通过QUIC协议与EMQX Broker进行通信。

通过这些组件的配置和集成，实验可以在分布式环境中顺利运行。

配置QUIC+MQTT所需环境

1. 在云服务器上安装docker：

```
# Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update

# Install docker
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-c
```

如果在云服务器上用的不是root用户名，可以把自己的用户名添加进docker组：

```
sudo groupadd docker # 如果已经存在就不管这行命令
sudo usermod -aG docker $USER
```

登出用户名再登入即可生效，后续运行docker相关的代码不需要再用sudo权限。

用如下代码验证docker是否安装及正确运行：

```
docker run hello-world
```

【Docker镜像加速】

如果docker无法拉取，即遇到如下错误:

```
Unable to find image 'hello-world:latest' locally
docker: Error response from daemon: Get "https://registry-1.docker.io/v2/": context dead
```

则需要配置Docker镜像加速:

```
sudo tee /etc/docker/daemon.json <<- 'EOF'
{
  "registry-mirrors": [
    "https://docker.mirrors.ustc.edu.cn",
    "https://docker.nju.edu.cn",
    "https://mirror.css.tencentyun.com",
    "https://do.nark.eu.org",
    "https://dc.j8.work",
    "https://docker.m.daocloud.io",
    "https://dockerproxy.com"
```

```
    ]
}
EOF
# 重启docker服务
sudo systemctl daemon-reload
sudo systemctl restart docker
```

重启Docker服务之后，即可正常拉取镜像

2. 在云服务器上运行broker：

```
docker run -d --name emqx \
  -p 1883:1883 -p 8083:8083 \
  -p 8084:8084 -p 8883:8883 \
  -p 18083:18083 \
  -p 14567:14567/udp \
  -e EMQX_LISTENERS__QUIC__DEFAULT__keyfile="/etc/certs/key.pem" \
  -e EMQX_LISTENERS__QUIC__DEFAULT__certfile="/etc/certs/cert.pem" \
  -e EMQX_LISTENERS__QUIC__DEFAULT__ENABLED=true \
  emqx/emqx:5.5.0
```

等待镜像下载并运行即可。命令运行成功后，可以用如下代码检查镜像是否运行：

```
docker ps
```

如果输出包含了刚才下载并运行的镜像，则docker成功运行。

3. 实验目录中提供了编译好的RDK上NanoMQ的二进制文件。在使用之前，须将依赖的库文件复制到系统目录：`sudo cp experiment4/libmsquic.so.2 /usr/lib/`

下面简要介绍基于NanoMQ的消息订阅和发布方法。

NanoMQ类似之前用过的ZeroMQ，也采取消息订阅和发布的模式。消息发布可在 `experiment/experiment4` 目录下运行如下命令：

```
./nanomq_cli pub -h <host_ip> -p <host_port> -t <topic> -q 2 -l --quic
```

其中，将 `<host_ip>` 替换为云服务器的VPN IP（如 `10.0.0.1`），将 `<host_port>` 替换为云服务器的quic监听端口，通常为14567，将 `<topic>` 替换为当前的话题。运行命令后，在命令行中输入需要发送的消息并回车，消息则会被发布到对应的话题。

消息订阅可在 `experiment/experiment4` 目录下运行如下命令：

```
./nanomq_cli sub -h <host_ip> -p <host_port> -t <topic> -q 2 -l --quic
```

其中，将 `<host_ip>` 替换为云服务器的VPN IP，将 `<host_port>` 替换为云服务器的quic监听端口，通常为14567，将 `<topic>` 替换为当前的话题。

在RDK上的命令行中分别运行订阅和发布程序，验证RDK和云服务器上的配置是否成功。

4. 在Python中，可以利用[Linux的pipe方法](#)调用 `nanomq_cli` 程序发布和订阅消息。

如果需要发布消息，可以在Python中调用 `print` 函数输出待发布的消息，再将消息pipe到nanomq_cli中。注意，`print` 函数需要打开 `flush` 选项。例如，如下名为 `print.py` 的python代码每隔1秒输出一个消息：

```
import time
for i in range(100):
    print("hello"+str(i), flush=True)
    time.sleep(1)
```

则为了发布上面所有的输出，可以在命令行中运行如下命令：

```
python3 print.py | ./nanomq_cli pub -h <host_ip> -p <host_port> -t <topic> -q 2 -l --quic
```

如果需要订阅消息，可以将nanomq_cli接收的消息pipe到Python代码中，并利用 `input` 函数读取。例如，如下名为 `input.py` 的python代码读入接收到的消息并输出：

```
for i in range(100):
    message = input()
    print(message)
```

在命令行中运行如下指令：

```
./nanomq_cli sub -h <host_ip> -p <host_port> -t <topic> -q 2 -l --quic | python3 input.p
```

注意：每次调用 `input` 函数则会阻塞目前的程序并等待输入，直到接收到指令。

利用上面的命令测试通过Python接收和发送消息。

基于VPN和QUIC+MQTT的打地鼠程序

编写python代码，将实验二的打地鼠程序移植到基于quic的nanomq框架上。

TCP和QUIC协议的对比

对比基于TCP和基于QUIC协议的通信速率。可以考虑增大负荷，观察发送和收到的时间差的区别。

可以使用参考代码中的 `test_timing.py`。

- 测试 TCP 协议（基于先前实验中使用的 ZeroMQ）的延迟的方法：
 - 在云服务器上运行实验二中的 `broker.py`
 - 在一台RDK上运行 `python3 test_timing.py <payload_size> zmq-tcp`，其中 `<payload_size>` 表示发送的数据包中除了头部外的字符数量，越大则负荷越大。例如：`python3 test_timing.py 100000 zmq-tcp`
 - 程序将每秒发送一个数据包并记录往返时间（单位：毫秒）
- 测试 QUIC 协议的延迟的方法：
 - 按上文所述，在云服务器上运行 EMQX Broker
 - 在一台RDK上运行 `./nanomq_cli sub -h 10.0.0.1 -p 14567 -t test -q 2 -l --quic | python3 test_timing.py <payload_size> quic | ./nanomq_cli pub -h 10.0.0.1 -p 14567 -t test -q 2 -l -quic`
 - 程序将每秒发送一个数据包并记录往返时间（单位：毫秒）

验收要求

- 完成VPN的搭建，保证Server和Client之间可以互相使用 `10.0.0.x` 的IP地址ping通。
- 完成QUIC+MQTT的配置，将先前实验中的打地鼠程序移植到基于QUIC的MQTT框架上。
- 进行TCP和QUIC协议的对比，对两者各收发多个不同 payload 大小的数据包、记录往返时间并进行对比。

选做任务：

- 考察不同因素对于通信延迟的影响。可变的因素包括：
 - TCP 协议 vs. QUIC 协议
 - 直接通信 vs. 通过 VPN 通信
 - 有线网 vs. 无线网
 - Payload 大小
 - 不同的 QoS level（`nanomq_cli` 中的 `-q` 参数），其功能参见[NanoMQ 说明文档](#)

参考

Quic-mqtt命令行工具[说明文档](#)

Comments